

# Technical Project Charter: Forgespec

**Team Members:** Ryan Stoffel, Payton Henry

**Submission Date:** Monday 4/13/26

## 1. Vision and Business Case

**Problem:** PC builders currently have no unified tool that combines part selection, real-time compatibility checking, AI-powered bottleneck analysis, and benchmark validation. Enthusiasts must cross-reference multiple sites to build a PC, identify performance issues, and verify that their hardware is performing as expected. This fragmented workflow wastes time and leads to poor purchasing decisions.

**Solution:** Forgespec is an AI-powered platform for PC building and benchmark analysis. Users search and select parts across all major categories (CPU, GPU, RAM, storage, motherboard, PSU, case, and cooler), and the system validates compatibility in real time as the build is assembled. Once a build is complete, Gemini analyzes it for bottlenecks and suggests better-value alternatives at the same price point. Users can also upload a benchmark screenshot, and the AI compares their actual performance against expected results for their exact hardware configuration, flagging anomalies. The user's saved build serves as a persistent context for all AI features, making every analysis specific to their hardware.

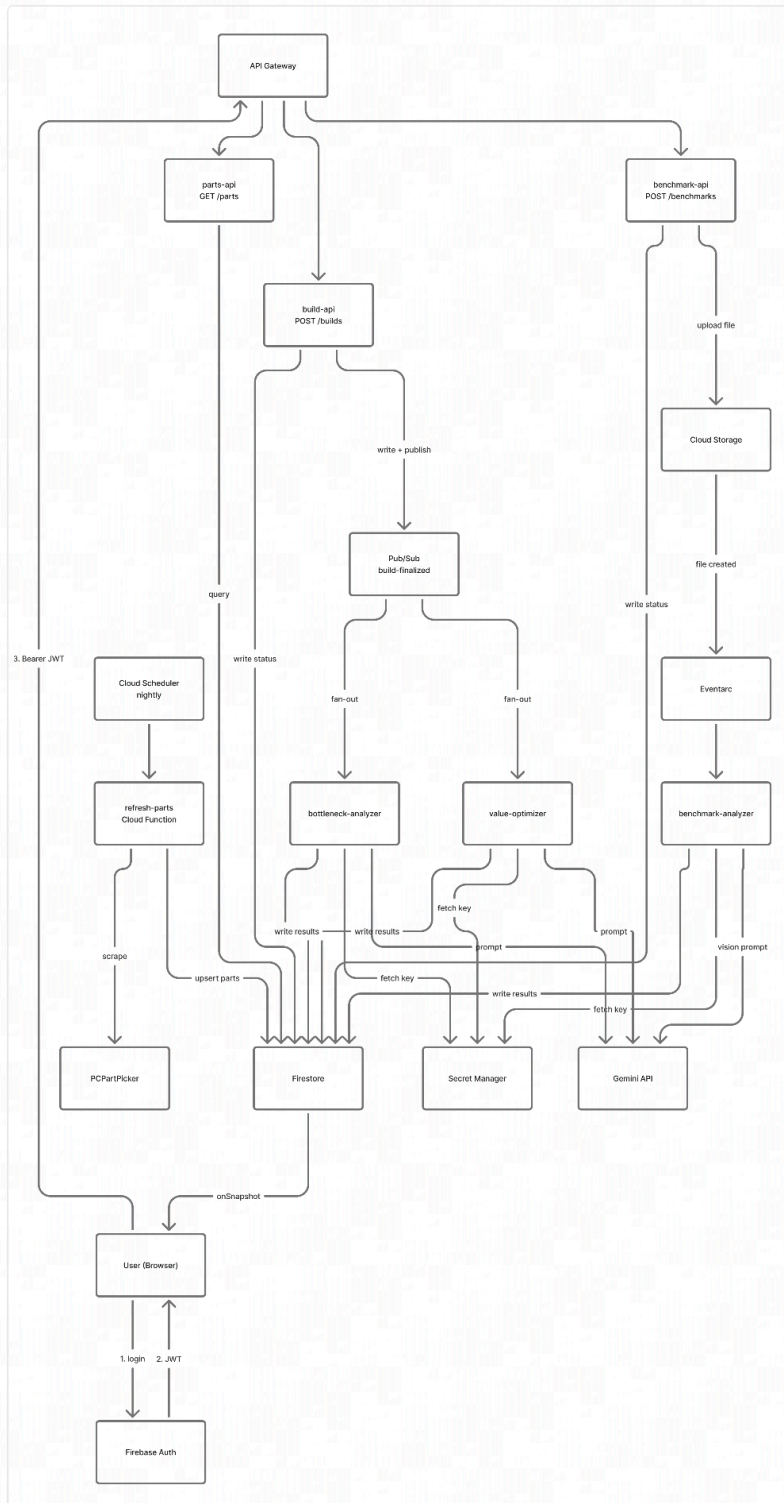
## 2. Technical Complexity Mapping

| Requirement Cluster          | Planned GCP Service                | Specific Role in the Application                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cluster 1: Edge and Auth     | API Gateway + Firebase Auth        | Users authenticate via Firebase Auth using email/password or Google OAuth. The Firebase JWT is included in every request Authorization header. API Gateway validates the JWT on every inbound request before forwarding traffic to any Cloud Run service. No backend service is reachable without a valid token. B2B access is supported via API Gateway API key validation as an alternative authentication method. |
| Cluster 2: Reactive Pipeline | Cloud Storage + Eventarc + Pub/Sub | Two non-HTTP event triggers drive the pipeline. First, when a user uploads a benchmark screenshot to Cloud Storage, Eventarc detects the new file and automatically triggers the benchmark processing pipeline. Second, when a build is finalized and written to Firestore, a                                                                                                                                        |

| Requirement Cluster                 | Planned GCP Service                                  | Specific Role in the Application                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     |                                                      | Pub/Sub message is published, splitting between the bottleneck analysis worker and the value-optimization worker in parallel, implementing a one-to-many distribution pattern.                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Cluster 3: Intelligence and Compute | Cloud Run + Cloud Functions + Gemini (Google AI API) | Three dedicated Cloud Run services handle heavy AI work: (1) a compatibility and bottleneck analyzer that takes the full build spec and returns a structured JSON report of performance bottlenecks and suggested swaps; (2) a benchmark analyzer that accepts a screenshot, extracts performance numbers via Gemini vision, and compares them against expected values for the user's build; (3) a value optimizer that proposes alternative parts at the same budget. Cloud Functions serve as lightweight connectors: triggering Pub/Sub on build finalization and routing Eventarc events from Cloud Storage to the appropriate Cloud Run service. |
| Cluster 4: Secure Orchestration     | Secret Manager + Cloud Scheduler + Cloud Functions   | All credentials are stored in Secret Manager and fetched at runtime: the Google AI API key, Firebase Admin SDK credentials, and any future third-party API keys. Each Cloud Run service and Cloud Function has its own dedicated service account with least-privilege bindings. Cloud Scheduler triggers a nightly Cloud Function that scrapes updated part pricing from PCPartPicker and upserts the Firestore parts catalog, keeping pricing data up to date without manual intervention.                                                                                                                                                           |
| Cluster 5: Persistence and State    | Firestore                                            | Firestore stores four top-level collections: parts (the full seeded parts catalog), users/{uid}/builds (saved build configurations and AI analysis results), users/{uid}/benchmarks (uploaded benchmark results and comparisons), and metrics/global (platform-wide counters). Atomic increments via FieldValue.increment() safely update the total builds analyzed counter under                                                                                                                                                                                                                                                                     |

| Requirement Cluster | Planned GCP Service        | Specific Role in the Application                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     |                            | concurrent load. A composite index on (uid, createdAt DESC) in the builds collection supports the dashboard listing query. A composite index on (uid, uploadedAt DESC) in the benchmarks collection supports benchmark history queries.                                                                                                                                                                                                                                                                                            |
| Cluster 6: UI       | Cloud Run (React frontend) | The React frontend is containerized and served from Cloud Run. Three primary user-facing flows are exposed: the Build Builder (part search, selection, and real-time compatibility feedback), the Build Analysis view (bottleneck report and suggested alternatives), and the Benchmark Upload view (drag-and-drop screenshot upload, real-time processing status via Firestore onSnapshot listener, and final comparison result). Users create accounts and log in via Firebase Auth on the frontend before making any API calls. |

### 3. Preliminary Architecture Diagram



### 1. End user sign-up flow

- a. Firebase Auth issues a JWT upon successful login or registration.
- b. The React frontend stores the JWT in memory and attaches it as a Bearer token in the Authorization header of every API request.
- c. API Gateway validates the JWT on every inbound request. Requests without a valid token are rejected with a 401 status code before reaching the backend.
- d. Only the user's UID and email are collected at registration. The application stores no passwords; Firebase Auth manages credentials.

### 2. Allowed user actions and endpoints

- a. GET /parts?category={category}&search={query} -- search the parts catalog
- b. POST /builds -- save or finalize a build, triggers AI analysis pipeline
- c. GET /builds/{buildId} -- retrieve a saved build and its analysis result
- d. POST /benchmarks -- upload a benchmark screenshot, triggers benchmark pipeline
- e. GET /benchmarks/{benchmarkId} -- retrieve benchmark comparison result
- f. All requests require Authorization: Bearer {jwt} header. Responses use standard JSON with appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 500).

### 3. Data pipeline

- a. Build finalization: React frontend POST /builds -> API Gateway -> build-api Cloud Run -> Firestore write -> Pub/Sub publish -> fan-out to bottleneck-analyzer and value-optimizer Cloud Run services -> results written back to Firestore.
- b. Benchmark upload: React frontend POST /benchmarks -> API Gateway -> benchmark-api Cloud Run -> file stored in Cloud Storage -> Eventarc trigger -> benchmark-analyzer Cloud Run -> Gemini vision analysis -> results written to Firestore -> frontend onSnapshot listener updates UI.
- c. Nightly refresh: Cloud Scheduler -> refresh-parts Cloud Function -> scrapes PCPartPicker -> upserts Firestore parts collection.

### 4. Public vs. private services

- a. Public (through API Gateway with JWT validation): /parts, /builds, /benchmarks endpoints.
- b. Private (no public ingress, invoked only by other GCP services): bottleneck-analyzer, value-optimizer, benchmark-analyzer Cloud Run services, and all Cloud Functions.

### 5. Service accounts (full CSV to be attached)

- a. sa-api-gateway: invokes Cloud Run API service only
- b. sa-build-api: reads/writes Firestore builds collection, publishes to Pub/Sub, writes to Cloud Storage
- c. sa-benchmark-api: reads/writes Firestore benchmarks collection, writes to Cloud Storage
- d. sa-bottleneck-analyzer: reads Firestore builds, reads Secret Manager (AI API key), calls Gemini API, writes Firestore
- e. sa-benchmark-analyzer: reads Cloud Storage, reads Secret Manager (AI API key), calls Gemini API, writes Firestore
- f. sa-value-optimizer: reads Firestore parts and builds, reads Secret Manager (AI API key), calls Gemini API, writes Firestore
- g. sa-refresh-parts: reads Cloud Storage, writes Firestore parts collection, reads Secret Manager

## 6. Secrets

- a. google-ai-api-key: accessed by sa-bottleneck-analyzer, sa-benchmark-analyzer, sa-value-optimizer
- b. firebase-admin-sdk: accessed by sa-build-api and sa-benchmark-api for server-side token verification

## 4. Scalability and Security Considerations

**Least Privilege Identity:** Every Cloud Run service and Cloud Function has a dedicated service account with only the IAM roles required for its specific task. No service account has Owner or Editor roles. API Gateway enforces JWT validation on every inbound request, so no backend service is reachable without authentication. Firestore security rules also enforce that users can only read and write documents whose UID field matches their authenticated identity, preventing cross-user data access. Build ownership is verified server-side before any AI analysis is triggered.

**Database Design:** Four Firestore collections: parts (catalog seeded from pc-part-dataset, refreshed nightly; fields: name, category, price, specs, url, updatedAt), users/{uid}/builds (fields: buildId, parts map, status, analysisResult, createdAt; composite index on uid + createdAt DESC), users/{uid}/benchmarks (fields: benchmarkId, buildId, uploadUrl, status, result, uploadedAt; composite index on uid + uploadedAt DESC), and metrics/global (totalBuildsAnalyzed counter using FieldValue.increment to prevent race conditions).

**Secret Strategy:** All API keys and service credentials are stored in Google Secret Manager. Secrets are never written to environment variables, code, or configuration files. Each Cloud Run service fetches its required secrets at startup using the Secret Manager API, authenticated via its dedicated service account. Secret versions are pinned to prevent rotation from causing unexpected failures.

**Response Times:** The React frontend never blocks while AI processing occurs. When a build is finalized or a benchmark is uploaded, the API returns immediately with a 202 Accepted status and Processing. The frontend uses a Firestore onSnapshot listener to receive real-time updates as the status field changes (Queued, Analyzing, Complete). This pattern ensures the UI remains responsive regardless of how long Gemini takes to process a request. The parts search endpoint reads directly from Firestore and is expected to respond in under 300ms under normal load.

## 5. Project Timeline

| Cluster                                        | Who Is Assigned | Estimated Date | Notes                                                                |
|------------------------------------------------|-----------------|----------------|----------------------------------------------------------------------|
| Cluster 5:<br>Firestore Schema<br>+ Parts Seed | Ryan            | 4/14/26        | Download the<br>PC-part-dataset JSON,<br>write a seed script, define |

| Cluster                                      | Who Is Assigned | Estimated Date | Notes                                                                                             |
|----------------------------------------------|-----------------|----------------|---------------------------------------------------------------------------------------------------|
|                                              |                 |                | Firestore collections and indexes                                                                 |
| Cluster 1: API Gateway + Firebase Auth       | Payton          | 4/16/26        | Firebase project setup, API Gateway config, JWT validation, test unauthenticated rejection        |
| Cluster 6: React Frontend (shell)            | Ryan            | 4/17/26        | Login/signup UI, build a builder with part search, connect to API Gateway                         |
| Cluster 4: Secret Manager + IAM + Scheduler  | Payton          | 4/18/26        | Create all service accounts, bind roles, store secrets, and deploy nightly refresh Cloud Function |
| Cluster 3: AI Cloud Run Services             | Both            | 4/21/26        | Deploy bottleneck-analyzer, benchmark-analyzer, value-optimizer; write and test Gemini prompts    |
| Cluster 2: Eventarc + Pub/Sub Pipeline       | Payton          | 4/22/26        | Wire Pub/Sub fan-out on build finalization, Eventarc trigger on benchmark upload                  |
| Cluster 6: Frontend Polish + End-to-End Test | Ryan            | 4/24/26        | Connect all UI flows, Firestore real-time listeners, error handling, and full end-to-end test     |
| Demo Video                                   | Both            | 4/25/26        | Record demo showing auth flow, build analysis, benchmark upload, and AI results                   |